UNIVERSITI MALAYA

UM Hackathon 2026

SYSTEM ANALYSIS DOCUMENT (SAD)

UM HACKATHON

DOMAIN	:	DOMAIN 2
DOMAIN NAME	:	AI FOR ECONOMIC EMPOWERMENT & DECISION INTELLIGENCE
GROUP NAME	:	MISTEAK
GROUP MEMBER	:	CELINE KUAN XIN YI
		LAW ZHI XUAN
		LOKE XIU XIANG
		TEVY CHONG TEIN VEY
		TEHVASHREE A/P PRABAKARA RAO

Table of Contents

1.0 Introduction....4
1.1 Purpose....5
1.2 Background....6
1.3 Target Stakeholder....7
2.0 System Architecture & Design....8
2.1 High Level Architecture....8
2.1.1 Overview....8
2.1.2 LLM as Service Layer....8
2.1.3 Dependency Diagram....9
2.1.4 Sequence Diagram....10
2.2 Technological Stack....11
2.2.1 Frontend....11
2.2.2 Backend....11
2.2.3 Database....12
2.2.4 Cloud/Deployment....12
2.3 Key Data Flows....13
2.3.1 Data Flows....13
2.3.2 Normalized Database Schema....15
3.0 Functional Requirements & Scope....16
3.1 Minimum Viable Product....16
3.1.1 Core features....16
3.2 Non-Functional Requirements (NFRs)....17
3.3 Out of Scope / Future Enhancements....18
4.0 Monitor, Evaluation, Assumptions & Dependencies....19
4.1 Technical Evaluation....19
4.1.1 Grayscale Rollout & A/B Testing....19

4.1.2 Emergency Rollback (ER) & Golden Release: 19
4.1.3 Priority Matrix 20
4.2 Monitoring 20
4.3 Assumptions 21
4.4 External Dependencies 22
5.0 Project Management & Team Contributions 23
5.1 Project Timeline 23
5.2 Team Members Role 24
5.3 Recommendations 25

1.0 Introduction

In the contemporary Food and Beverage (F&B) industry, many managers still rely on random guesswork to boost their business growth rather than a systematic approach, primarily due to disconnected operational, sales, and customer metrics. This lack of centralized data forms a major obstacle in capturing demand trends and designing pricing strategies, thereby hindering businesses from evolving in the long-term. Furthermore, non-technical users such as entrepreneurs and business owners struggle to find the massive time and effort required to learn specialized analytical skills for business analysis. However, current business intelligence tools lack user-friendliness and cost-effectiveness, necessitating technical expertise and funding that SMEs often lack. To bridge this gap, Misteak serves as a novel solution to this industry-wide challenge, seamlessly translating raw data into clear strategic direction, empowering stakeholders to confidently make data-driven decisions.

1.1 Purpose

The system analysis document highlights the technical scope, architectural framework and design-related decisions behind the development of Misteak.

So, the key elements of system that has been covered are as followed:

1. **Architecture**

   Misteak is introduced as a unified data pipeline and decision intelligence platform in a Python environment. It utilizes Streamlit as the frontend interface, encompassing data preprocessing, reasoning supported by the Z.AI GLM, and recommendation generation within a centralized environment.

2. **Data Flows**

   Structured restaurant dataset (csv or xlsx) is ingested, cleaned, transformed and engineered through the Python backend. Afterwards, the preprocessed data is used to calculate module scores and further passed to the Z.AI GLM model to perform context-aware reasoning, generate insights and provide data-driven recommendations. Finally, the insights flow back to the Streamlit frontend, displayed as visual dashboards, smart alerts and AI recommendations.

3. **Model Process:**

   Misteak follows an end-to-end pipeline where raw data is cleaned, processed, and transformed into KPIs and scores. These are sent to the Z.AI GLM for reasoning, generating insights and recommendations. The results are then validated and displayed in the Streamlit interface for decision-making.

4. **Role of Reference:**

   This document serves as a technical guide outlining system architecture, data flow, and design decisions. It supports development, debugging, and future enhancements while ensuring consistency and clarity across the system for easier maintenance and scalability.

1.2 Background

Before Misteak, the typical approach for F&B companies to analyse their business performance was manual operations including spreadsheet analysis and manual calculations. Analysis was largely intuitive and prone to inconsistencies and errors. They lacked formal decision support, relying on the interpretation of numerical information alone. Anomaly detection (for example, detecting a sudden decrease in sales) was usually done in hindsight and relied on visual inspection. And scalability was constrained by the individual's analytical skills, meaning it was hard to scale or formalise decision-making.

Misteak is a revolutionary leap from this status quo. It provides a data processing pipeline that uses Python-based feature engineering to convert raw data into metrics. Insight is improved using a hybrid approach to scoring and reasoning that incorporates AI, delivering structured and interpretable conclusions. The system incorporates decision support, providing actionable insights with contextual intelligence. The system also performs anomaly detection through a Smart Alerts System that detects anomalies in near real-time. In summary, the system is modularised to allow it to grow beyond the limits of expert knowledge and to be easily enhanced.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Restaurant Owner	Views business insights and make strategic decisions	Clear, actionable recommendations with minimal technical complexity
Manager	Monitors performance metrics and responds to alerts	Real-time anomaly detection and performance driver visibility
Decision Maker	Uses AI-generated recommendations to support decision-making in pricing and business strategy.	Accurate and data-driven insights, along with actionable recommendations for pricing
Entry-Level Analyst	Uses system to generate and interpret insights	Easy-to-understand dashboards with pre-engineered analytical outputs
Development Team	Builds and maintains the platform	Well-documented architecture, clear API contracts, modular codebase
QA Team	Validates system behaviour across user roles	Defined scope, documented request/response formats, edge case coverage

2.0 System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

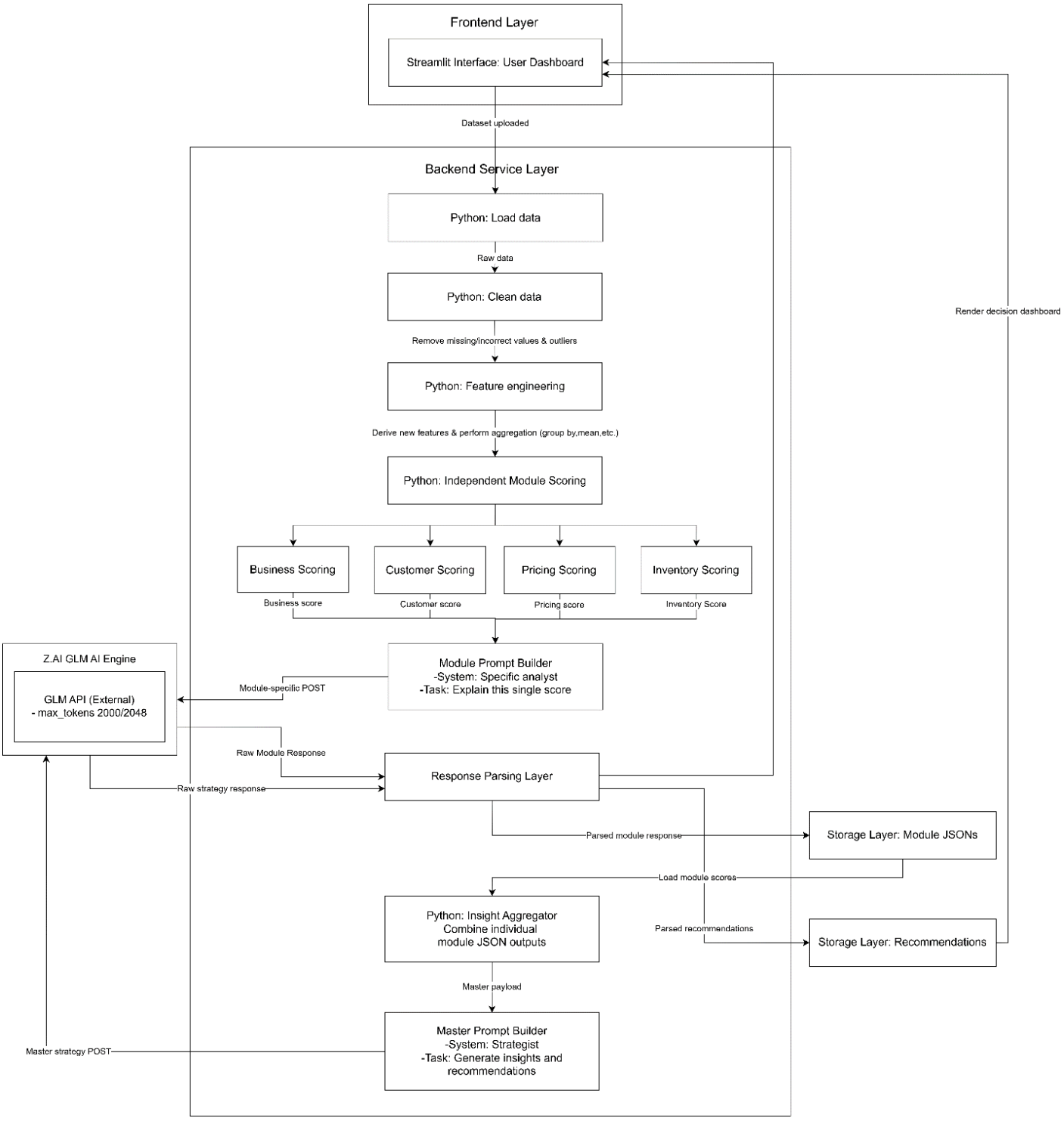
Type	Details
System	Web-based Analytics Dashboard
Architecture	Modular Monolithic (Streamlit-based) with AI Service Layer

Misteak is realised as an analytics and decision intelligence web-based system with a modular monolithic architecture. The system is built with Python 3.10 and uses Streamlit for the front-end to build interactive dashboards and multipage apps. The system core makes use of the Z.AI General Language Model (GLM) reasoning engine, through an API. Data is stored in lightweight file formats like CSV, XLSX (for the input dataset) and JSON (for intermediate structured data). The system architecture is divided into four main layers. The presentation layer includes the Streamlit interface, displaying dashboards, notifications, simulations and AI-recommended actions. The processing layer consists of Python modules for data transformation, feature engineering, and scoring. The AI layer communicates with the Z.AI GLM API, such as prompt generation and response interpretation. The data layer is responsible for datasets and storing the results.

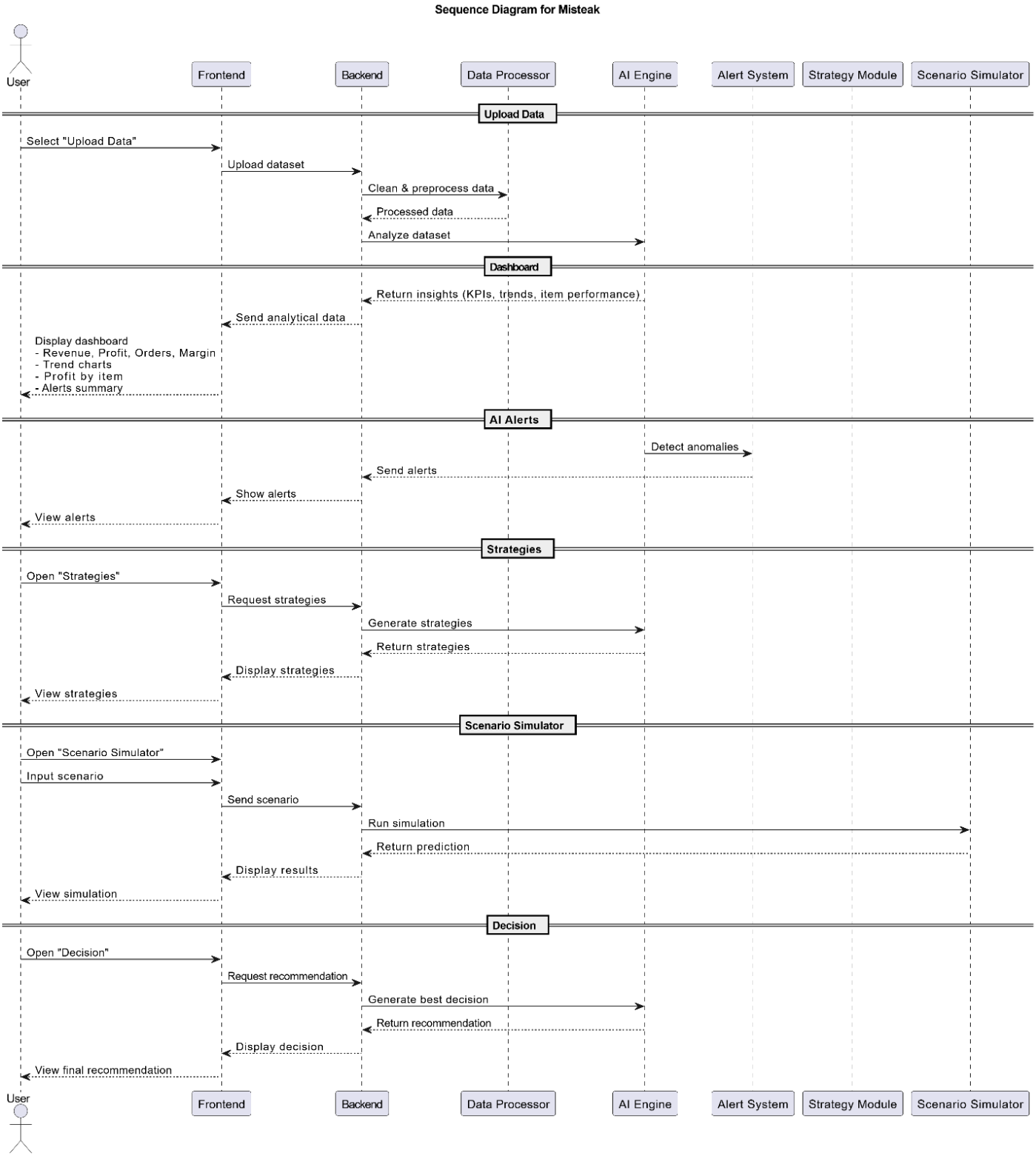
2.1.2 LLM as Service Layer

The Z.AI GLM layer of the system is an external service that provides advanced reasoning, improving decision-making. The process starts with data preprocessing to clean and standardise the data. Performance metrics and scoring are derived through algorithms. This data is then packaged into a JSON object, providing contextual information to the model. A prompt featuring structured data and templates is then sent to the GLM API. The model interprets this to provide explanations, recommendations and confidence scores. These are then filtered and sorted and displayed to the user in relevant parts of the interface, such as in dashboards or strategy areas.

2.1.3 Dependency Diagram



2.1.4 Sequence Diagram



2.2 Technological Stack

2.2.1 Frontend

The frontend of Misteak is developed using Streamlit, a Python-based framework that enables rapid creation of interactive web applications without the need for a separate frontend technology stack such as React or Angular. Streamlit is particularly suitable for data-driven applications, as it provides built-in support for visualisation components, state management, and multi-page navigation. In this system, it is used to design and render all user-facing modules, including dashboards, alerts, strategy recommendations, and simulation interfaces. The framework allows seamless integration with Python processing logic, enabling real-time updates of charts and insights as soon as data is processed. This significantly reduces development complexity while ensuring that non-technical users can interact with the system through an intuitive and responsive interface.

2.2.2 Backend

The backend logic of Misteak is implemented entirely in Python 3.10+, which serves as a unified environment for data processing, business logic, and AI integration. Instead of a traditional client-server backend architecture, the system adopts a modular monolithic structure where processing modules are directly invoked within the application runtime. Key functionalities in this layer include data preprocessing, feature engineering, KPI calculation, and the Pricing Score Engine, which evaluates business performance across multiple dimensions. Additionally, this layer handles communication with the Z.AI GLM API, including prompt construction, payload structuring, and response parsing. By centralising all logic in Python, the system achieves high maintainability and faster development cycles, making it ideal for a hackathon prototype while still being extensible for future enhancements.

2.2.3 Database

Misteak currently utilises a lightweight file-based storage approach, relying on CSV and XLSX files for input data and JSON format for structured intermediate outputs. This design choice simplifies data handling and eliminates the overhead associated with managing a full-scale database system during early development. Processed data, scoring outputs, and AI payloads are primarily handled in memory during each session, ensuring fast access and minimal latency. While this approach is efficient for prototyping and demonstration purposes, it is designed with future scalability in mind. The system can be extended to incorporate a relational database such as SQLite for local persistence or PostgreSQL for production environments, enabling features like historical data tracking, multi-user support, and improved data integrity.

2.2.4 Cloud/Deployment

The current deployment strategy for Misteak focuses on simplicity and accessibility, utilising local environments and Streamlit Cloud for hosting and demonstration. This approach allows quick deployment with minimal configuration, making it ideal for hackathon settings where speed and reliability are critical. However, the system architecture is designed to be cloud-ready, meaning it can be easily migrated to more robust cloud platforms such as AWS or Google Cloud in future iterations. Potential deployment enhancements include using virtual machines for scalability, implementing load balancing for handling concurrent users, and integrating cloud storage solutions for persistent data management. This forward-compatible design ensures that Misteak can evolve from a prototype into a production-ready system with minimal architectural changes.

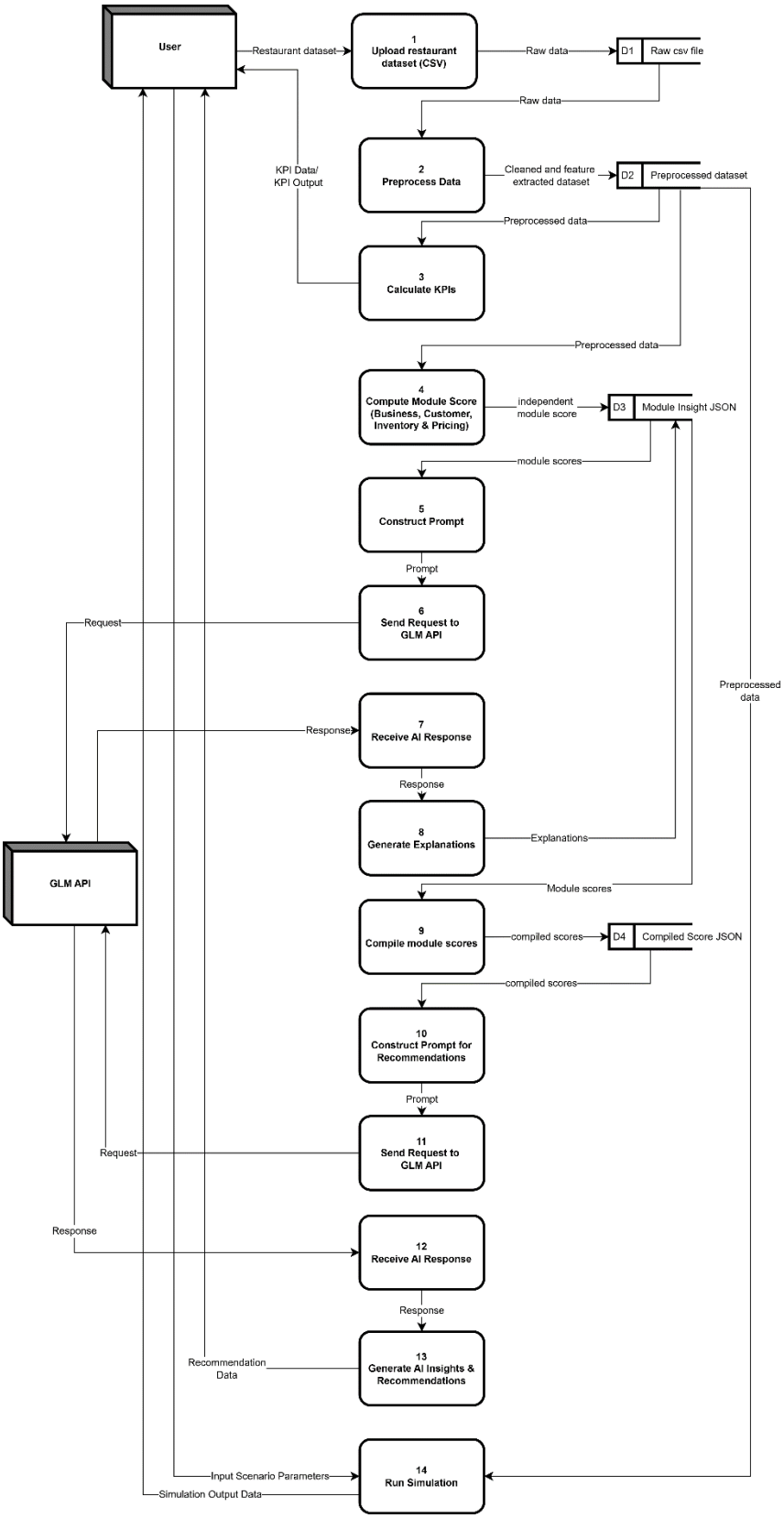
2.3 Key Data Flows

2.3.1 Data Flows

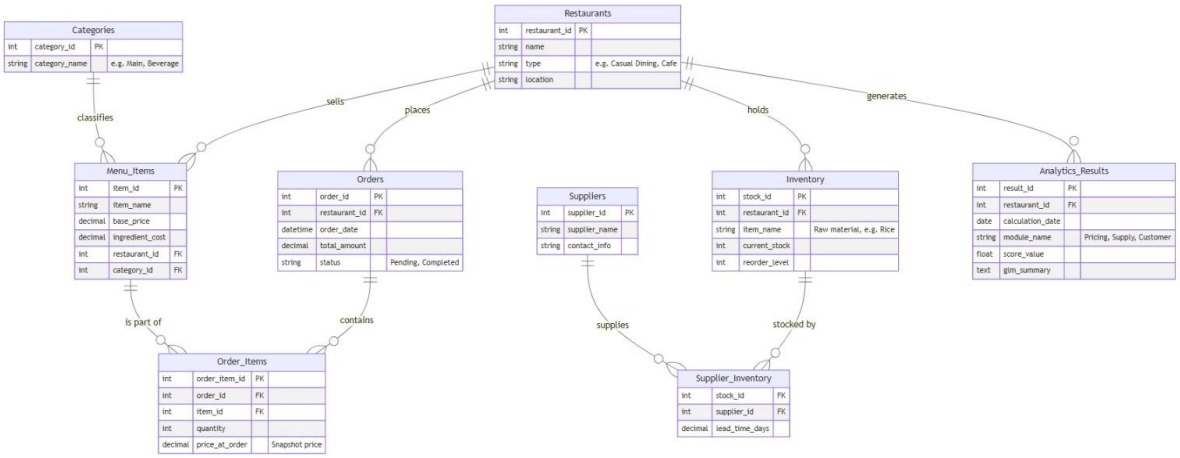
The main data flow begins when a user uploads a restaurant dataset. The application validates file type and schema, cleans the data, computes derived features, generates KPIs and scores, constructs an AI payload, receives AI insights, and renders results in the Streamlit interface.

Flow	Input	Process	Output / Storage
Flow 1: Data Processing	CSV/XLSX dataset.	Validate schema, handle missing values, normalize columns, remove invalid rows.	Cleaned in-memory dataframe and optional processed CSV/JSON.
Flow 2: Dashboard Analytics	Cleaned dataset and aggregated features.	Calculate revenue, profit, trends, and other KPIs.	Dashboard charts and KPI cards.
Flow 3: Scoring	KPIs, features, and business rules.	Run pricing, customer, supply, and business score engines.	Structured score JSON.
Flow 4: AI Decision Support	Scores, KPIs, and context.	Generate prompt, call Z.AI GLM, parse and validate response.	Recommendations, explanations, and strategy panels.
Flow 5: Simulation	User-selected scenario parameters.	Recalculate affected metrics and regenerate AI context.	Projected impact and decision guidance.

2.3.1.1 Data Flow Diagram (DFD)



2.3.2 Normalized Database Schema



3.0 Functional Requirements & Scope

3.1 Minimum Viable Product

3.1.1 Core features

No	Features	Description
1	Dashboard Analytics	Provides a comprehensive overview of business performance, including key performance indicators (KPIs) such as revenue, profit and trends, helping users to quickly understand the trends and patterns.
2	AI Decision Support	Recommends the most optimal business strategy and provides quantifiable impact along with detailed explanations.
3	Strategy Recommendations	Provides different business strategies based on current data, where each strategy is assessed with different risk levels (e.g., low, medium, high) to support informed decision-making.

3.2 Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Performance	AI recommendation completes within 10 seconds	API call to ilmu-glm-5.1 model with max_tokens limited to 1200–2048
Reliability	Fallback message is triggered if API is unavailable	try/except block in inventory_module_scores.py catches API errors and uses a default summary
Usability	UI navigable by non-technical restaurant owners	Streamlit multipage layout with clear labels, KPI cards, and visual charts via Plotly
Maintainability	Module scores is independently maintained and updated	Each module (calc_pricing_score.py , inventory_module_scores.py and restaurant_pipeline.ipynb) runs as a standalone script
Data Integrity	Ensure data is cleaned before scoring to remove noise	warnings.filterwarnings('ignore') and fillna() used in preprocessing scripts to handle missing values

3.3 Out of Scope / Future Enhancements

- **Real-time database integration**
  The system relies on flat-file CSV files, rather than capable of interacting with real-time database integrations or live telemetry streaming.

- **Multi-user collaboration**
  This platform is currently designed for single primary user, temporarily unable to support seamless multi-collaborators, shared organizational workspaces or Role-Based Access Control.

- **Mobile application version**
  Misteak is currently compatible with web browser only, tablet or phone version is not yet supported.

- **Backend API deployment**
  The system is currently unavailable for external API connections.

- **Integration with POS systems**
  Misteak currently operates as a standalone web application and lacks dedicated API integrations with external POS ecosystems or third-party restaurant management software.

- **Production-grade scalability**
  The current architecture is not yet optimized to handle vast volume of concurrent user traffic and distributed load balancing.

4.0 Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

4.1.1 Grayscale Rollout & A/B Testing

- **Grayscale Rollout**

  Novel core features like updated AI prompt or modifications to the mathematical module scoring logic will initially be released to a pilot group of 5%-10% of users. The development team will continuously monitor the backend error and API success rates before completely rollout to all users.

- **A/B Testing**

  To improve user experience, A/B testing will be conducted on how AI recommendations should be visually presented on the Streamlit frontend. For example, Variant A may render AI recommendations as a detailed, data-heavy analytical table, while Variant B focuses on streamlined, concise representations like “Smart Cards”. The development team will monitor the time spent on that page to determine which layout drives faster for decision-making.

4.1.2 Emergency Rollback (ER) & Golden Release:

When there are critical production anomalies like Pandas data pipeline crashing during CSV ingestion or Z.AI API connection timeouts, an Emergency Rollback is triggered.

The Github Actions CI/CD Pipeline will automatically stop the current deployment and rollback to the last stable Golden Release, which refers to the most recent commit that successfully passed the 90% regression threshold and attained a 100% pass rate on critical data ingestion and JSON output tests.

4.1.3 Priority Matrix

- P1 Critical: If the system fails to ingest standard CSV files, or Z.AI API fails to return a valid JSON format for more than 5% of requests, emergency roll back to Golden release.

- P2 High: If mathematical logic encounters bugs such as pricing scores calculation result is out of the boundary between 0.0 and 1.0 or AI hallucinations occur, the system is patched via Pull request and pushed through the automated PyTest pipeline within 24 hours.

- P3 Medium: If UI issues like rendering failure of Streamlit dashboard but is not as critical as can crash the app, this issue will be logged in the team's internal issue tracking system and will be fixed in next updates.

4.2 Monitoring

- **External API Health**
  Continuously monitoring the Z.AI GLM API response time and timeout rates, ensuring not exceeding 800ms.

- **Data Pipeline Integrity**
  Overseeing the Pandas backend and identifying whether unhandled exceptions or schema validation errors frequently occurred during the ingestion of user-uploaded CSVs.

- **AI Parsing Success**
  Tracking the response of the JSON parser to ensure the AI returns valid JSON and immediately logging a "Parsing Error" if unstructured text is returned instead.

Trigger Action:
If API latency exceeds 800ms consistently or JSON parsing fails on more than 5% of inputs, the automated alerts will be triggered and sent to the development team to investigate potential AI rate-limiting or prompt drift.

4.3 Assumptions

a. Users can provide structured or semi-structured restaurant data in CSV or XLSX format.

b. Uploaded datasets contain enough information to calculate meaningful KPIs such as revenue, sales volume, pricing, profit, or demand trends.

c. Users have stable internet access when the system needs to call the Z.AI GLM API.

d. The application is used by a limited number of users during the hackathon prototype demonstration.

e. Human review is still required before implementing major business decisions based on AI recommendations.

f. The Z.AI GLM API remains available and API credentials are configured securely.

g. The MVP does not require real-time POS integration, multi-user authentication, or enterprise-level concurrency.

4.4 External Dependencies

Tools	Purpose	Risks
Z.AI GLM API	It can understand the underlying patterns of data and infer it based on the context. Apart from that, it provides various alternatives and suggests the best strategy and explains it with detailed insights for confident decision-making.	High: the entire system highly relies on GLM reasoning. If GLM faces bottlenecks or does not response, the restaurant business analysis will fail to be performed.
Streamlit	Provides the web interface, dashboard, navigation, and user input components.	Medium: not ideal for high-concurrency production deployments.
Python Libraries	Support data processing, charts, scoring, and API communication.	Medium: dependency version changes may break modules.
CSV/XLSX Input Files	Primary data source for the MVP.	Medium: inconsistent schemas or poor-quality data can reduce accuracy.
Cloud Hosting / Internet	Hosts application and enables API communication.	Medium: downtime or network issues may affect system availability.

5.0 Project Management & Team Contributions

5.1 Project Timeline

Phase	Days	Activities
Planning & Architecture	Day 1–2	Define project scope, identify problem statement, finalise technology stack, design system architecture, assign team roles, and prepare initial data schema.
Data Pipeline & Scoring	Day 3–4	Develop data preprocessing pipeline, implement feature engineering logic, and build the Pricing Score Engine to evaluate business performance metrics.
Core UI Development	Day 5–6	Design and implement Streamlit multi-page interface, including Dashboard, Alerts, Strategy, and Simulation pages, along with visualisations.
AI Integration	Day 7–8	Integrate Z.AI GLM API, design prompt templates, construct structured payloads, and implement fallback logic for reliability.
Testing & Refinement	Day 9	Conduct end-to-end testing using sample datasets, handle edge cases, improve UI/UX, and optimise performance.
Demo Preparation	Day 10	Finalise documentation, prepare demo flow, rehearse presentation, and ensure system stability for submission.

5.2 Team Members Role

Member	Role	Responsibilities
Member A	Project Leader / Frontend Developer/ Documentation	Handles project coordination tasks, develops the Streamlit user interface including dashboard layouts and navigation, and prepares key sections of the documentation to ensure clarity and consistency.
Member B	AI Engineer / Data Engineer / Documentation	Designs and implements prompt structures for the Z.AI GLM API, develops data preprocessing and feature engineering components, and documents AI workflows and system logic.
Member C	AI Engineer / Data Engineer / Documentation	Builds data transformation pipelines, constructs structured JSON payloads for AI processing, refines prompt inputs, and documents data flow and model behaviour.
Member D	AI Engineer / Data Engineer / Backend	Develops core system logic including the Pricing Score Engine, manages data flow between modules, and ensures backend processing and AI integration operate correctly.
Member E	Frontend Developer / QA Engineer / Backend	Implements user interface components in Streamlit, performs system testing and validation across features, and ensures smooth integration between frontend and backend modules.

5.3 Recommendations

- **Implement Caching Mechanism**

  Introducing a caching layer will allow the system to store frequently generated AI responses and processed data. This reduces repeated calls to the Z.AI GLM API, improving response time and lowering token costs. It is especially useful when users explore similar scenarios multiple times within a session.

- **Adopt Persistent Database**

  Moving from file-based storage (CSV/JSON) to a database system will improve data management and reliability. Persistent storage enables session history, long-term trend analysis, and better data integrity, making the system more suitable for real-world usage beyond a prototype.

- **Add User Feedback Mechanism**

  Incorporating a simple feedback system, such as rating AI recommendations, allows the collection of user validation data. This feedback can be used to improve prompt design and enhance the accuracy and usefulness of future recommendations.

- **Enhance Performance Optimization**

  Optimising data processing and aggregation pipelines will ensure the system remains responsive even with larger datasets. Techniques such as pre-aggregation, efficient data structures, and session-based caching can help maintain performance standards.

- **Enable Real-Time Data Integration**

  Integrating real-time data sources, such as POS systems, would allow the platform to provide live insights and alerts. This enhancement would significantly increase the system's value by enabling immediate decision-making based on current business conditions.